Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

LEARNER GUIDE

For

Create RESTful APIs and Web Apps with Python Flask

TGS Ref No: TGS-2021010365

Conducted by

Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

Version 11.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
10.0	Prior release	Legacy Flask courseware retained as the content-coverage reference.	Dr Alfred Ang
11.0	5 September 2026	Rebuilt with FastAPI, OpenAPI 3.1, Northstar Commerce SQLite/CRM/order labs, VS Code and evidence-led AI prompts.	Dr Alfred Ang

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Course Overview

This two-day course uses FastAPI and OpenAPI 3.1 to build Northstar Commerce: a responsive catalog, CRM and ordering web app backed by SQLite. Every script begins from a structured vibe-coding prompt, is written to an isolated CodeBundle, reviewed in VS Code, tested and accepted only with evidence.

Learning Outcomes

- LO1: Identify and assess FastAPI as middleware for creating connections between web clients, applications and data stores.
- LO2: Integrate data and functions into a browser web app through typed FastAPI routes and reusable Python services.
- LO3: Support API-level integration using REST semantics, JSON and an OpenAPI 3.1 contract.
- LO4: Perform tests and checks on FastAPI-to-SQLite connections using Swagger UI, pytest, logs and VS Code debugging.
- LO5: Modify a FastAPI service to improve integration, validation, compatibility, performance and security.

Environment Setup

Step 1: Install Python 3.11 or later and Visual Studio Code.

Step 2: Open VS Code Extensions (Ctrl/Cmd+Shift+X) and install Python, Pylance, REST Client, SQLite Viewer and YAML.

Step 3: Open the selected lab folder and choose Python: Select Interpreter -> .venv.

Step 4: Create a virtual environment: python3 -m venv .venv

Step 5: Activate it: source .venv/bin/activate (macOS/Linux) or .venv\Scripts\activate (Windows).

Step 6: Install FastAPI and the OpenAI Python SDK: python -m pip install -r working-files/requirements.txt

Step 7: Preview the prompt without an API call: python ai_vibe.py --dry-run

Step 8: Export OPENAI_API_KEY in the shell and set OPENAI_MODEL to a model available to your account; never store either value in lab files.

Step 9: Generate the review bundle: python ai_vibe.py

Step 10: Review working-files/generated/ and copy only accepted changes into working source.

Step 11: Run the service: cd working-files && uvicorn app.main:app --reload

Step 12: Open http://127.0.0.1:8000/docs and execute GET /health/live.

Topic 01: REST API Fundamentals and AI Vibe Coding

Contracts · resources · HTTP semantics · OpenAI Python SDK · prompt-to-test workflow

The API contract is the product boundary

A request crosses a boundary; the contract makes method, path, data and outcomes explicit.

- Client owns intent
- Route owns transport
- Service owns business rules
- Repository owns persisted state

Artifact: GET /api/v1/products?status=open → 200 + JSON array

Failure to diagnose: A prompt that says only ‘build an API’ leaves every contract decision implicit.

Verification: A reviewer can predict the request and response without reading implementation code.

Model resources before routes

Use nouns for resources and HTTP methods for operations.

- Collection: /api/v1/products
- Member: /api/v1/products/{product_id}
- Filter: ?status=open
- Avoid verbs such as /getProducts

Artifact: Resource map: Product {id,sku,name,price,stock_quantity,status}

Failure to diagnose: Action-style URLs multiply and weaken predictable semantics.

Verification: Every route maps to one resource state transition.

HTTP methods encode intent

GET reads, POST creates, PUT replaces, PATCH changes part, DELETE removes.

- GET is safe
- PUT is idempotent
- POST is normally non-idempotent
- PATCH needs an explicit partial-update rule

Artifact: Method × state-change matrix

Failure to diagnose: Using POST for every action hides semantics from clients and tools.

Verification: Repeat an idempotent request and confirm the resulting state is unchanged.

Status codes are machine-readable outcomes

Status codes separate transport outcome from response detail.

- 2xx success

- 4xx client correction
- 5xx server investigation
- Body carries actionable detail

Artifact: Response distribution from a 100-request test run

Failure to diagnose: Returning 200 for failures breaks monitoring and client control flow.

Verification: Success, validation, missing-resource and conflict paths each assert a specific code.

FastAPI turns types into runtime contracts

Python annotations drive parsing, validation, editor help and OpenAPI schemas.

- Path values are converted
- Pydantic validates bodies
- Return types filter responses
- OpenAPI is generated

```
@app.get("/api/v1/products/{product_id}")
def read_product(product_id: int) -> ProductRead:
    return service.get(product_id)
```

Artifact: Typed route → validation → schema

Failure to diagnose: Untyped dictionaries defer errors until later code paths.

Verification: Send an invalid type and inspect the structured 422 error location.

The ASGI request lifecycle

Uvicorn accepts the connection, FastAPI resolves the route and dependencies, then serializes the response.

- socket
- middleware
- router
- dependencies
- handler
- response model

Artifact: Browser → Uvicorn → FastAPI → service → JSON

Failure to diagnose: Blocking work in an async handler can stall concurrent requests.

Verification: Trace one request using a request ID across access and application logs.

Vibe coding needs a closed verification loop

Prompting accelerates drafting; tests and review decide whether generated code is acceptable.

- Specify
- Generate
- Run

- Inspect
- Refine
- Record evidence

Artifact: Prompt + acceptance checks + diff

Failure to diagnose: Accepting plausible code without executing it transfers uncertainty into production.

Verification: Every prompt names inputs, outputs, constraints and tests; every change produces evidence.

A good prompt is an executable design brief

Constrain framework, paths, schemas, errors, persistence and tests before asking AI to code.

- Weak: build a product API
- Strong: FastAPI, /api/v1/products, SQLite, validation, 404/409, pytest, no ORM

Artifact: Prompt contract checklist

Failure to diagnose: Over-specific implementation prompts can freeze a poor design; specify behaviour first.

Verification: Generated endpoints match a written route table and pass named tests.

The Python SDK turns a prompt into a typed file bundle

A reproducible vibe-coding run records the model, prompt, mock context and structured output before any code is accepted.

- OpenAI() reads OPENAI_API_KEY
- responses.parse sends the brief
- CodeBundle constrains file paths and contents
- generated/ keeps AI output separate for review

```
client = OpenAI()
rsp = client.responses.parse(
    model=os.environ["OPENAI_MODEL"],
    input=messages, text_format=CodeBundle
)
bundle = read_bundle(rsp)
```

Artifact: ai_vibe.py + Prompts.pdf + mock-data.json

Failure to diagnose: Copying unstructured model text directly over working source hides provenance and can overwrite trusted code.

Verification: The SDK produces a validated CodeBundle in generated/; pytest and human review decide whether to apply it.

Lab 01: Prompt a REST API Contract

Maps to A1, A2. Convert Northstar Commerce requirements into a versioned OpenAPI-first route and data contract.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Create a behaviour-first OpenAPI 3.1 contract for Northstar Commerce using FastAPI and SQLite. Define Product, Customer, Order and OrderItem schemas; list collection/member routes, request and response JSON, 200/201/204/401/404/409/422 outcomes, and at least six acceptance tests. Do not write implementation code yet. Return only api-contract.md, openapi-draft.yaml and machine-readable test cases.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Install the Microsoft Python, Pylance, REST Client, SQLite Viewer and Red Hat YAML extensions in VS Code.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Open the scenario and identify Product, Customer and Order resources plus their relationships.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Create api-contract.md with collection and member routes under /api/v1.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Draft OpenAPI 3.1 paths, schemas, responses and API-key security in openapi-draft.yaml.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Ask the assistant to critique ambiguous identifiers, money fields, stock rules and order state transitions.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Validate the YAML and save the accepted contract decisions with the prompt evidence.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

The contract covers products, customers and orders, names 200/201/204/401/404/409/422 outcomes and validates as OpenAPI 3.1.

Evidence to Submit

- api-contract.md and openapi-draft.yaml
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 02: Run the First FastAPI Service

Maps to A1, A4. Create and verify a typed product endpoint and its generated OpenAPI documentation.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Generate the smallest typed FastAPI service with GET /health, GET /api/v1/products and GET /api/v1/products/{product_id}. Use Pydantic response models, integer path validation and OpenAPI summaries. Add TestClient tests for health, route inventory and a non-integer product id returning structured 422 JSON.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Create and activate a virtual environment.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Install the packages from requirements.txt.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Use the VS Code Python: Select Interpreter command to choose .venv.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Open app/main.py and inspect the FastAPI application metadata.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Run uvicorn app.main:app --reload from the working-files folder.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Open /docs and execute GET /health/live.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Add GET /api/v1/products/{product_id} with product_id typed as int.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Send a non-integer ID and capture the 422 response as evidence.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 15: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 16: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

GET /health/live returns 200; /docs lists the product route; invalid path input returns structured 422 JSON.

Evidence to Submit

- main.py with /health and /api/v1/products routes
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Topic 02: AI-Assisted API Functions, Routing and Security

Routers · dependency injection · reusable services · CORS · API keys

Separate transport from business logic

Thin route functions translate HTTP; service functions express rules independently of the web framework.

- Router: parse/return
- Service: decisions
- Repository: SQL
- Schema: validate/serialize

Artifact: router → service → repository

Failure to diagnose: SQL embedded in every route creates duplication and fragile tests.

Verification: The service can be unit-tested without starting Uvicorn.

APIRouter creates bounded route modules

A router groups paths, tags and dependencies under a shared prefix.

- prefix=/api/v1/products
- tags=[products]
- shared dependencies
- included once by app

```
router = APIRouter(prefix="/api/v1/products", tags=["products"])
app.include_router(router)
```

Artifact: products.router mounted by main.py

Failure to diagnose: Duplicate prefixes and inconsistent tags produce confusing OpenAPI output.

Verification: OpenAPI lists every product endpoint once under the intended tag.

Dependency injection makes controls reusable

Depends() resolves shared behaviour before the handler and passes the verified result in.

- read header
- validate token
- open DB
- call handler
- close resource

Artifact: Depends(require_api_key)

Failure to diagnose: Copy-pasted authentication diverges route by route.

Verification: Protected routes reject missing/invalid keys and accept the configured key.

API-key authentication is a boundary control

A demo API key illustrates authentication, but production needs rotation, secure storage and least privilege.

- X-API-Key header
- constant-time comparison
- environment configuration
- 401/403 policy

```
def require_api_key(x_api_key: str = Header()):
    if not secrets.compare_digest(x_api_key, settings.api_key):
        raise HTTPException(401, "Invalid API key")
```

Artifact: Header → dependency → authorised principal

Failure to diagnose: Hard-coded secrets in source or frontend are publicly exposed.

Verification: Secret scan finds no live key; unauthorised requests fail before business logic.

CORS is a browser origin policy

Origin is scheme + host + port; the browser checks whether frontend JavaScript may read the response.

- http://localhost:5500
- http://localhost:8000
- different ports = different origins
- allow only required origins

Artifact: Preflight OPTIONS → CORS headers

Failure to diagnose: Wildcard origins cannot safely support credentials and widen exposure.

Verification: Browser devtools shows the allowed origin and successful preflight.

Security requirements are testable behaviour

Translate ‘secure the endpoint’ into explicit threat, control and verification rows.

- Threat: unauthorized write
- Control: API key dependency
- Failure: 401
- Evidence: test + log

Artifact: Threat-control-test matrix

Failure to diagnose: A control with no negative test is an unverified assumption.

Verification: Each security requirement has at least one rejection test.

Review AI-generated security code adversarially

Generated code is a draft; inspect secrets, validation, SQL, error detail, CORS and dependency versions.

- Plausible: hard-coded key
- Acceptable: environment key + ignored .env
- Plausible: f-string SQL
- Acceptable: parameters

Artifact: Security review diff

Failure to diagnose: AI may optimise for a working demo rather than operational safety.

Verification: A checklist review produces an approved diff and recorded exceptions.

Sync and async must match the work

Async improves concurrency for awaitable I/O; blocking SQLite calls belong in normal def handlers or a threadpool.

- async def for awaitable clients
- def for blocking libraries
- measure, do not cargo-cult
- keep transactions short

Artifact: Latency under 1 vs 20 concurrent requests

Failure to diagnose: Calling blocking work directly inside async def stalls the event loop.

Verification: A concurrency test keeps p95 latency within the chosen threshold.

Lab 03: Split Routes and Reuse a Service Function

Maps to A3, A8. Refactor a monolithic FastAPI file into router and service modules without changing the contract.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Refactor the supplied FastAPI product service into thin routes and reusable service functions. Mount one APIRouter at /api/v1/products, preserve the existing contract and add regression tests proving every OpenAPI operation appears once. Explain module ownership in concise docstrings.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Run the baseline tests and record the passing count.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Move product decision logic into service.py.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Create an APIRouter with prefix /api/v1/products and tag products.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Include the router once in main.py.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Run the original tests and compare /openapi.json before and after.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Remove duplicate imports and document module ownership in README.md.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

All baseline tests still pass and OpenAPI exposes each route exactly once.

Evidence to Submit

- main.py, routes.py and service.py
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 04: Add API-Key Protection and CORS

Maps to A7, A8. Implement a reusable API-key dependency and an explicit browser-origin allow list.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Add a reusable X-API-Key dependency to product, customer and order write routes and configure CORS for only the origin supplied in mock-data.json. Read secrets from environment variables, compare safely, never place a live key in source or browser JavaScript, and test missing, invalid and valid credentials.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Copy .env.example to .env and set a local demo key; never commit .env.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Implement require_api_key() using the X-API-Key header.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Attach the dependency to POST, PATCH and DELETE routes.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Configure CORSMiddleware for the exact frontend development origin.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Test missing, invalid and valid keys with curl or Swagger UI.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Inspect the browser preflight request and response headers.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

Unauthorized writes return 401; a valid key succeeds; only the configured origin receives CORS permission.

Evidence to Submit

- security.py plus protected write routes
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Topic 03: Request Handling, Data Validation and JSON Responses

Path/query/body data · Pydantic models · response filtering · PATCH

FastAPI classifies inputs by declaration

A name in the path is a path parameter; a scalar is query data; a Pydantic model is a JSON body.

- /products/{product_id}
- ?status=open
- ProductCreate body
- Header for cross-cutting metadata

Artifact: Input-source map

Failure to diagnose: Ambiguous or duplicated fields create contradictory contracts.

Verification: OpenAPI shows every input in the intended location.

Pydantic validates at the boundary

Field types and constraints reject invalid data before business logic runs.

- min_length
- pattern
- ge/le
- Literal/Enum
- custom model validator

```
class ProductCreate(BaseModel):
    sku: str = Field(pattern=r"^[A-Z0-9-]+$")
    price: Decimal = Field(ge=0, decimal_places=2)
    stock_quantity: int = Field(ge=0)
```

Artifact: ProductCreate schema

Failure to diagnose: Validation after SQL can persist invalid state.

Verification: Boundary tests cover invalid SKU, negative price/stock and unsupported status.

422 errors point to the exact invalid field

The error payload includes location, message and error type for each failed input.

- loc: body.sku
- msg: string pattern mismatch
- type: string_pattern_mismatch
- client can map error to UI

Artifact: Structured validation error JSON

Failure to diagnose: Replacing 422 with a generic 400 discards useful machine-readable detail.

Verification: Frontend renders the failing field and keeps the user-entered values.

Response models are an output firewall

FastAPI validates and filters returned data against the declared output model.

- handler returns object
- response_model validates
- private fields removed
- JSON encoded

Artifact: ProductRow → ProductRead

Failure to diagnose: Returning database rows directly can leak internal columns.

Verification: A test asserts secret/internal fields never appear in JSON.

JSON shape is part of compatibility

Clients depend on field names, types, optionality and nesting—not just values.

- Stable: {items,total}
- Breaking: rename title→name
- Compatible: add optional field
- Version breaking contracts

Artifact: Before/after response diff

Failure to diagnose: Silent schema drift breaks frontend code at runtime.

Verification: Contract tests compare responses to the documented schema.

PATCH requires an explicit missing-value rule

model_dump(exclude_unset=True) distinguishes omitted fields from fields explicitly set to null.

- omitted = keep current
- present = update
- null = allowed only where schema permits
- validate merged state

```
changes = patch.model_dump(exclude_unset=True)
updated = service.update(product_id, changes)
```

Artifact: Partial update dictionary

Failure to diagnose: Using model defaults overwrites fields the client did not send.

Verification: Updating one field leaves all other stored fields unchanged.

Pagination protects service and client

Limit and offset bound work; response metadata lets the UI navigate predictably.

- limit default 20
- cap limit at 100
- offset >= 0

- include total count

Artifact: Payload size by page limit

Failure to diagnose: Unbounded collection routes can exhaust memory and freeze browsers.

Verification: A request above the cap is rejected or constrained as documented.

Serialize dates and enums deliberately

Use typed datetime and enum fields so JSON encoding and OpenAPI remain consistent.

- datetime → ISO 8601
- Enum → stable string
- decimal policy
- timezone decision

Artifact: Python value ↔ JSON value map

Failure to diagnose: Naive local timestamps become ambiguous across clients.

Verification: Round-trip tests preserve meaning across encode and decode.

Lab 05: Validate Product and Customer Data

Maps to A3, A5. Define typed product and customer schemas with field-level and cross-field constraints.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Create Pydantic v2 Product, Customer, Order and OrderItem input/output models. Constrain SKU, email, Decimal price, stock quantity and status; distinguish required, optional and omitted values. Add schema examples and boundary tests that assert the exact 422 error locations for invalid mock requests.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Define product status as active, inactive or discontinued.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Add SKU, product-name, decimal price and stock-quantity constraints.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Create separate product and customer input, patch and output models.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Execute valid and invalid requests in /docs.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Record each 422 loc, msg and type value.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Add examples that appear in the generated OpenAPI schema.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

Invalid SKU, price, stock, email and status values are rejected before repository code; valid JSON becomes a typed model.

Evidence to Submit

- schemas.py with ProductCreate, ProductPatch and ProductRead
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 06: Filter Responses and Implement PATCH

Maps to A4, A8. Use response models and exclude_unset semantics for safe partial updates.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Revise the FastAPI product routes so response models exclude internal fields and PATCH uses model_dump(exclude_unset=True). Preserve unspecified fields, define the empty-patch rule and add tests for one-field updates, response filtering, missing products and stable JSON shapes.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Add an internal field to the repository record and confirm it is not in ProductRead.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Declare response_model on list, create and update routes.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Use model_dump(exclude_unset=True) for partial updates.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Patch only stock_quantity and verify SKU, name, price and status remain unchanged.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Send an empty PATCH and apply the documented no-op rule.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Add a regression test for response-field filtering.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

PATCH changes only supplied fields and no internal database field appears in the JSON response.

Evidence to Submit

- PATCH /api/v1/products/{id} with stable ProductRead output
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Topic 04: API Error Handling, Testing and Debugging

HTTPException · logs · TestClient · isolation · failure diagnosis

Errors are designed outcomes

Expected client failures use HTTPException; unexpected failures are logged and mapped without leaking internals.

- 404 missing
- 409 state conflict
- 422 validation
- 500 unexpected

Artifact: Failure taxonomy

Failure to diagnose: Catching every Exception and returning 200 hides defects.

Verification: Each documented failure has a deterministic status and response body.

Raise HTTPException at the decision point

Stop processing when a resource or rule fails and return a meaningful detail.

- lookup
- decide
- raise
- framework serializes

```
product = repo.get(product_id)
if product is None:
    raise HTTPException(404, "Product not found")
```

Artifact: 404 response path

Failure to diagnose: Returning None may later cause a confusing serialization or attribute error.

Verification: Missing ID returns 404 and performs no write.

Custom handlers standardise error shape

An exception handler converts different internal failures into a consistent public envelope.

- exception
- handler
- request_id
- public code
- log full detail

Artifact: {error:{code,message,request_id}}

Failure to diagnose: Leaking stack traces exposes file paths and implementation details.

Verification: Clients parse one stable error envelope while logs retain diagnostic context.

Tests follow Arrange–Act–Assert

Arrange state, perform one HTTP interaction, then assert transport and business evidence.

- Arrange test DB
- Act with TestClient
- Assert status
- Assert JSON
- Assert persisted state

Artifact: pytest test case

Failure to diagnose: A test that asserts only status may miss corrupt data.

Verification: Each write test checks response and database state.

TestClient exercises the ASGI application

FastAPI's TestClient sends requests without a live network server.

- same routes
- same validation
- fast deterministic loop
- pytest-friendly

```
client = TestClient(app)
r = client.post("/api/v1/products", json={"sku":"NS-MUG-02","name":"Field Notes
Mug","price":"26.00"})
assert r.status_code == 201
```

Artifact: client.post('/api/v1/products', json=...)

Failure to diagnose: Manual Swagger checks are useful exploration but not regression protection.

Verification: The suite runs unattended and reports repeatable pass/fail evidence.

Isolate test data

Tests need a temporary SQLite database and dependency override so runs cannot alter demo data.

- temporary path
- create schema
- override dependency
- run test
- remove file

Artifact: production DB dependency ↔ test override

Failure to diagnose: Tests against the real database are destructive and order-dependent.

Verification: A test run leaves the demo database checksum unchanged.

Debug from evidence, not guesses

Follow request ID, status, log event, stack location, input and database state in order.

- reproduce
- minimise

- inspect logs
- set breakpoint
- fix
- add regression test

Artifact: Failure-to-fix trace

Failure to diagnose: Changing multiple things at once destroys causal evidence.

Verification: The regression test fails before the fix and passes after it.

A test portfolio covers multiple risks

Balance happy paths, validation, not-found, conflict and authorization tests.

- functional
- boundary
- negative
- security
- regression

Artifact: Test distribution by risk

Failure to diagnose: A suite dominated by happy paths creates false confidence.

Verification: Every route has at least one success and one failure assertion.

Lab 07: Standardise Errors and Request Logs

Maps to A5, A7. Create predictable error responses and trace a request from status to log event.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Generate request-ID middleware, structured request logging and one public error envelope for 404 and 409 outcomes. Logs may include method, path, status, request id and elapsed time but no API key or body secret. Add tests that correlate the response request id with captured log evidence.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Add request-ID middleware that accepts or creates X-Request-ID.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Log method, path, status and elapsed time without logging secrets.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Raise 404 for missing products and 409 for duplicate titles.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Add an exception handler for the public error envelope.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Trigger each error and match the response request_id to the log.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Confirm stack traces are absent from public JSON.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

404 and 409 use the same envelope; each response request ID locates one log trace; secrets are absent.

Evidence to Submit

- error envelope and request-ID middleware
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 08: Test and Debug the API

Maps to A5, A6, A7. Build a pytest regression suite, reproduce a seeded defect and verify the fix.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Using the seeded failing cases in mock-data.json, produce a focused pytest regression suite and a debug plan. Reproduce the failure with the smallest request, assert response and persisted state, identify the likely boundary, and propose the minimum patch without weakening existing acceptance tests.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Run pytest -q and capture the seeded failing test.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Read the assertion diff, request payload and response body.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Set a breakpoint in the failing service function.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Reproduce with the smallest single request.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Fix the root cause without changing unrelated behaviour.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Add a regression assertion, rerun the full suite and save the output.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

The seeded test fails before the fix; the full suite passes after it; the regression assertion remains.

Evidence to Submit

- tests/test_commerce.py with success and failure coverage
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Topic 05: API Architecture, Data Models and Database Integration

Layered design · SQLite · parameterized SQL · transactions · CRUD

Layering localises change

Schemas, routes, services and repositories change for different reasons.

- schemas.py: contracts
- routes.py: HTTP
- service.py: rules
- db.py: SQL
- main.py: composition

Artifact: Five-file application boundary

Failure to diagnose: A monolithic app.py entangles tests and makes AI edits broad and risky.

Verification: A schema change touches the smallest justified set of files.

SQLite is a serverless relational engine

The database is a local file accessed by the Python process; it still enforces tables, keys and transactions.

- single file
- ACID transactions
- SQL queries
- excellent demo/local fit
- limited write concurrency

Artifact: northstar.db with products table

Failure to diagnose: Treating SQLite as an unstructured file loses relational guarantees.

Verification: Schema inspection shows primary key, defaults and constraints.

Design the table around invariants

Use constraints and defaults to protect state even when a code path is wrong.

- Product and Customer keys
- OrderItem foreign keys
- money/stock CHECK
- unique SKU/email
- created_at default

Artifact: Commerce relational schema

Failure to diagnose: Application-only validation can be bypassed by another writer.

Verification: Invalid direct SQL writes fail at the database boundary.

Parameterized SQL separates code from data

Use placeholders and parameter tuples; never concatenate user input into SQL.

- query template fixed
- values bound separately
- quotes handled safely
- injection payload stays data

```
row = conn.execute(
    "SELECT * FROM products WHERE id = ?", (product_id,)
).fetchone()
```

Artifact: SELECT ... WHERE id = ?

Failure to diagnose: f-string SQL allows input to change query structure.

Verification: An injection-shaped product name is stored or rejected as data, never executed.

Transactions define atomic state changes

A write either commits as a complete unit or rolls back.

- BEGIN
- validate current state
- INSERT/UPDATE
- COMMIT
- ROLLBACK on error

Artifact: Request-scoped connection context

Failure to diagnose: Partial writes leave contradictory state when an exception interrupts work.

Verification: A forced failure leaves the database in its pre-request state.

CRUD maps contracts to SQL

Each endpoint has a predictable statement, success code and missing-resource path.

- POST→INSERT→201
- GET→SELECT→200/404
- PATCH→UPDATE→200/404
- DELETE→DELETE→204/404

Artifact: Endpoint-to-SQL matrix

Failure to diagnose: A DELETE that always returns 204 cannot distinguish an absent record.

Verification: CRUD integration tests cover both existing and missing IDs.

Indexes trade write work for read speed

An index on frequently filtered columns reduces scanning as rows grow.

- index status
- measure query plan
- avoid indexing every field
- small demos may not need many

Artifact: Measured query time by row count

Failure to diagnose: Premature indexes add write overhead and complexity without evidence.

Verification: EXPLAIN QUERY PLAN shows the expected index for the target query.

Connection lifetime is an ownership decision

Open a short-lived request-scoped connection, configure row access and foreign keys, then close it reliably.

- Context manager
- row_factory
- PRAGMA foreign_keys=ON
- commit/rollback
- close

Artifact: Connection lifecycle

Failure to diagnose: A leaked connection or long transaction holds locks and creates intermittent failures.

Verification: Load tests show no growing open-resource count and no locked-database errors.

Lab 09: Design the Commerce CRM Database

Maps to A1, A3. Create constrained products, customers, orders and order_items tables with relational integrity.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Design SQLite products, customers, orders and order_items tables with primary/foreign keys, UNIQUE email/SKU, money and stock checks, order-state checks and indexes. Generate an idempotent init_db function using sqlite3.Row and PRAGMA foreign_keys=ON, plus tests that initialise twice and reject invalid relationships.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Draw the Product-Customer-Order-OrderItem relationship and list its invariants.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Create schema.sql with primary keys, foreign keys, NOT NULL, UNIQUE, defaults and CHECK constraints.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Implement init_db() to execute the schema safely and repeatedly.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Enable sqlite3.Row and foreign key enforcement on each connection.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Inspect the schema and indexes using the SQLite CLI or Python.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Attempt an invalid order-item insert and record the foreign-key or stock constraint failure.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

Initialisation is idempotent and SQLite enforces unique SKU/email, valid money/stock, order status and foreign keys.

Evidence to Submit

- db.py and schema.sql
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 10: Implement SQLite CRUD and Orders Safely

Maps to A3, A4, A5. Connect product, customer and order operations to parameterized SQL and atomic transactions.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Implement parameterized SQLite repository functions for product/customer CRUD and atomic order creation. Validate stock inside the same transaction, insert order items, decrement inventory or roll back fully, use bounded pagination and an allow-list for PATCH columns. Add CRUD, insufficient-stock and injection-shaped-input tests.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Implement create_product with placeholders and return the inserted row.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Implement list_products with bounded limit/offset and optional status filter.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Implement create_customer and reject duplicate email addresses.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Implement create_order as one transaction that checks stock, inserts order rows and decrements stock.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Implement dynamic PATCH SQL from a validated allow-list of fields.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Implement delete_product and inspect rowcount.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run CRUD and injection-shaped-input tests against a temporary database.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 15: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.

Evidence to Submit

- repository.py with catalog, CRM and order transaction functions

- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Topic 06: REST API Documentation, Integration and Deployment

OpenAPI · browser fetch · static frontend · versioning · packaging

OpenAPI is an executable interface inventory

FastAPI derives paths, parameters, schemas, responses and security metadata from code.

- /openapi.json
- /docs Swagger UI
- /redoc
- client generation
- contract review

Artifact: Code annotations → OpenAPI → interactive docs

Failure to diagnose: Undocumented exceptions and security rules make docs misleading.

Verification: A reviewer can execute every public operation from /docs.

Operation metadata improves usability

Tags, summaries, descriptions, response codes and examples turn generated docs into a usable contract.

- tags
- summary
- status_code
- responses
- examples

```
@router.post("/", response_model=ProductRead, status_code=201,
    summary="Create a product", responses={409:{"description":"Duplicate"}})
```

Artifact: Annotated create-product operation

Failure to diagnose: Generic function names and missing response descriptions force guesswork.

Verification: The docs distinguish create, validation, conflict and authorization outcomes.

The browser client uses fetch as an API adapter

JavaScript serializes JSON, sends HTTP, checks status, parses JSON and renders DOM state.

- form event
- fetch
- response.ok
- response.json
- render

- error banner

Artifact: HTML form → JS adapter → FastAPI → SQLite

Failure to diagnose: Calling response.json() before checking 204 or non-JSON errors causes secondary failures.

Verification: Network panel and visible UI agree on status and payload.

Keep the frontend state derived from API responses

Render the server-returned product instead of assuming the requested change succeeded.

- disable during request
- use response body
- refresh list
- show error detail
- preserve input on failure

Artifact: UI state transition model

Failure to diagnose: Optimistic updates without rollback leave the page inconsistent.

Verification: Simulated 409/500 responses do not corrupt the visible product list.

Same-origin hosting removes demo CORS friction

Mount static files in FastAPI for a single-origin demo; configure explicit CORS only when origins differ.

- Same origin: / + /api
- Separate dev origin: 5500 + 8000
- Explicit allow list
- never put secret in browser

Artifact: Deployment topology comparison

Failure to diagnose: A frontend API key embedded in JavaScript is public by definition.

Verification: Browser loads assets and API calls without blocked preflight or exposed secrets.

Health endpoints separate liveness from readiness

Liveness says the process responds; readiness proves required dependencies can serve traffic.

- /health/live → process
- /health/ready → database SELECT 1
- fast and dependency-light
- no sensitive details

Artifact: Health-check contract

Failure to diagnose: A single ‘OK’ endpoint can report healthy while the database is unavailable.

Verification: Stopping database access changes readiness but not liveness.

Package a reproducible release

Pin dependencies, ignore secrets/data, initialise schema deterministically and document one start command.

- requirements.txt
- .env.example
- .gitignore
- schema init
- tests
- README
- uvicorn command

Artifact: Release bundle checklist

Failure to diagnose: Shipping a local .env or northstar.db leaks secrets and test data.

Verification: A clean clone installs, tests and starts using only documented steps.

API versioning manages breaking change

Use /api/v1 for a stable contract and introduce v2 only when compatibility cannot be preserved.

- additive change first
- deprecation window
- usage telemetry
- migration guide
- retirement date

Artifact: Client adoption across a deprecation window

Failure to diagnose: Changing v1 response fields in place silently breaks consumers.

Verification: Both versions pass their contract suites until v1 retirement.

Lab 11: Connect the Commerce Web App

Maps to A3, A4, A6. Use fetch to power a responsive dashboard for products, customers and orders through FastAPI.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Generate an accessible responsive HTML/CSS/JavaScript Northstar Commerce dashboard that uses fetch with the supplied FastAPI contract. Implement product/customer creation, order placement, KPI refresh and inventory rendering without page reload; keep live secrets out of committed code; render structured errors and empty/loading/failure states.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Run the completed API and open the root demo page.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Inspect loadProducts() and trace GET /api/v1/products in the Network panel.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Wire product and customer forms to POST JSON and render the returned records.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Place an order and confirm the dashboard updates order count, revenue and stock.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Display structured API errors in the accessible status region.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Test desktop and narrow-screen layouts plus empty, loading, validation and server-error states.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

The browser reads/writes only through FastAPI, SQLite state changes correctly and the UI remains usable at 375px and desktop widths.

Evidence to Submit

- static/index.html, styles.css and app.js
- One successful request/response or automated test result
- One diagnosed failure and its corrected result

Lab 12: Document and Bundle the Capstone

Maps to A4, A6, A8. Produce a clean, testable release bundle with API docs, frontend assets and operational instructions.

Prompt and SDK Assets

- ai_vibe.py - OpenAI Python SDK runner with typed CodeBundle output
- mock-data.json - authorised synthetic context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - exact first-run and refinement prompts
- working-files/generated/ - isolated AI draft output

First-Run Prompt

Review the complete FastAPI/SQLite/browser project and generate release documentation only. Include clean setup, test, run and verification commands; OpenAPI and environment notes; a bundle inventory; and a checklist excluding .env, databases, caches and virtual environments. Do not invent passing evidence.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
cd working-files && pytest -q
```

Detailed Step-by-Step Procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 7: Run the full test suite and save the terminal evidence.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 8: Review /docs and add missing summaries, tags, response descriptions and examples.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 9: Verify .gitignore excludes .env, *.db, caches and virtual environments.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 10: Create .env.example without live secrets.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 11: Follow README setup from a clean virtual environment.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 12: Zip only the documented source, tests and static assets.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 13: Run the acceptance checklist and record the final version identifier.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 14: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Step 15: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the stated action. Observe the current request, response, file or log state. Compare it with the acceptance criterion before continuing. Save a screenshot or terminal excerpt that proves the state.

Acceptance Criteria

A clean copy installs, tests, starts, serves the storefront/CRM dashboard, completes catalog/customer/order flows and contains no secret or database file.

Evidence to Submit

- release-ready Northstar Commerce API
- One successful request/response or automated test result

- One diagnosed failure and its corrected result

Reference Links

FastAPI: Modern Python Web Development: Bill Lubanovic, O'Reilly, 2024 - supplied ebook

Web API Development with Python: Rehan Haider, 2021 - supplied ebook

FastAPI Tutorial: https://fastapi.tiangolo.com/tutorial/

FastAPI First Steps: https://fastapi.tiangolo.com/tutorial/first-steps/

Request Body: https://fastapi.tiangolo.com/tutorial/body/

Response Models: https://fastapi.tiangolo.com/tutorial/response-model/

Handling Errors: https://fastapi.tiangolo.com/tutorial/handling-errors/

SQL Databases: https://fastapi.tiangolo.com/tutorial/sql-databases/

Testing: https://fastapi.tiangolo.com/tutorial/testing/

Security: https://fastapi.tiangolo.com/tutorial/security/first-steps/

CORS: https://fastapi.tiangolo.com/tutorial/cors/

Deployment: https://fastapi.tiangolo.com/deployment/

VS Code FastAPI Tutorial: https://code.visualstudio.com/docs/python/tutorial-fastapi

OpenAPI Learn: https://learn.openapis.org/

OpenAPI Specification 3.1.2: https://spec.openapis.org/oas/v3.1.2.html

OpenAI Python SDK: https://github.com/openai/openai-python

OpenAI Structured Outputs: https://platform.openai.com/docs/guides/structured-outputs